AKS + ACR Lab: Flask + MySQL App with Autoscaling

Note: the GitHub repo could not be read automatically and nothing can be run in your Azure subscription, so a real final URL cannot be given here. The URL appears at Step 8 as `http://<EXTERNAL-IP>`, where the IP is assigned by Azure to your own cluster.

This lab assumes the common Flask + MySQL app, which listens on port 5000, uses `requirements.txt`, and connects to MySQL. Step 2 has a quick check, and the lines to adjust are marked **ADJUST**.

Architecture

```
Internet -> Azure Load Balancer (public IP)
         -> Service (LoadBalancer :80)
         -> Flask Pods (2-10 replicas, HPA on CPU)
         -> Service (ClusterIP mysql :3306)
         -> MySQL Pod (1 replica, Azure Disk PVC)

Image: built and stored in ACR, pulled by AKS via managed identity
```

Step 0: Prerequisites

You need an Azure subscription, plus Azure CLI and kubectl. Easiest is Azure Cloud Shell (https://shell.azure.com, Bash), which already has everything. Otherwise:

```
az login
az account set --subscription "<your-subscription-id-or-name>"
az aks install-cli        # installs kubectl if missing
```

Set the variables, using a globally unique lowercase alphanumeric ACR name:

```
export RG=flask-lab-rg
export LOCATION=centralindia
export ACR_NAME=flasklabacr$RANDOM
export AKS_NAME=flask-lab-aks
export IMAGE=flask-mysql-app
export TAG=v1
```

If this is a new subscription, register the providers:

```
az provider register --namespace Microsoft.ContainerService
az provider register --namespace Microsoft.ContainerRegistry
az provider register --namespace Microsoft.Compute
```

Step 1: Create the Resource Group, ACR and AKS

```
# Resource group
az group create -n $RG -l $LOCATION

# Container registry
az acr create -g $RG -n $ACR_NAME --sku Basic

# AKS cluster, attached to ACR, with the node autoscaler enabled
az aks create \
  -g $RG -n $AKS_NAME \
  --node-count 2 \
  --node-vm-size Standard_B2s \
  --enable-managed-identity \
  --attach-acr $ACR_NAME \
  --enable-cluster-autoscaler --min-count 2 --max-count 4 \
  --generate-ssh-keys
```

`--attach-acr` gives the cluster's kubelet identity the `AcrPull` role, so you don't need image pull secrets. `--enable-cluster-autoscaler` lets AKS add nodes when pods can't be scheduled. This takes 5-10 minutes.

Connect kubectl and verify:

```
az aks get-credentials -g $RG -n $AKS_NAME --overwrite-existing
kubectl get nodes
```

Both nodes should show `Ready`.

Step 2: Clone the Repo and Inspect It

```
git clone https://github.com/ajit010/Python-Flask-MySQL-App.git
cd Python-Flask-MySQL-App
ls
cat app.py | head -40
cat requirements.txt
```

Check four things:

1. **How does the app connect to MySQL?** Look for `MYSQL_DATABASE_HOST` or `localhost`. If it's hardcoded, change it to read environment variables (**ADJUST:** use your repo's actual config key names):
2. **Which port does it run on?** Usually `app.run()` on 5000. For containers it must bind `0.0.0.0`, which gunicorn handles below.
3. **Which database objects does it need?** Tables or stored procedures (see Step 5).
4. **What is the Flask object called?** Usually `app` in `app.py`, which gives gunicorn the target `app:app`.

```
import os
app.config['MYSQL_DATABASE_USER'] = os.getenv('DB_USER', 'root')
app.config['MYSQL_DATABASE_PASSWORD'] = os.getenv('DB_PASSWORD', '')
app.config['MYSQL_DATABASE_DB'] = os.getenv('DB_NAME', 'BucketList')
app.config['MYSQL_DATABASE_HOST'] = os.getenv('DB_HOST', 'localhost')
```

Step 3: Dockerfile

If the repo has no Dockerfile, create one:

```
cat > Dockerfile <<'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt gunicorn
COPY . .
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
EOF
```

If `requirements.txt` is missing or incomplete, create it. The tutorial-style app needs:

```
Flask
flask-mysql
PyMySQL
cryptography
werkzeug
```

ADJUST: `app:app` must match your file and Flask variable name.

Step 4: Build the Image in ACR

No local Docker is needed. ACR builds it in the cloud:

```
az acr build -r $ACR_NAME -t $IMAGE:$TAG .
az acr repository show-tags -n $ACR_NAME --repository $IMAGE -o table
```

You should see `v1` listed.

Step 5: Deploy MySQL

```
kubectl create namespace flask-app
```

5.1 Secret

```
kubectl create secret generic mysql-secret -n flask-app \
```

```
  --from-literal=MYSQL_ROOT_PASSWORD='ChangeMe#123' \
  --from-literal=MYSQL_DATABASE='BucketList'
```

5.2 Database init script

This is for the common BucketList-style app. **ADJUST:** if your repo has its own `.sql` file, use that instead.

```
cat > init.sql <<'EOF'
CREATE DATABASE IF NOT EXISTS BucketList;
USE BucketList;

CREATE TABLE IF NOT EXISTS tbl_user (
  user_id BIGINT NOT NULL AUTO_INCREMENT,
  user_name VARCHAR(45) NULL,
  user_username VARCHAR(45) NULL,
  user_password VARCHAR(255) NULL,
  PRIMARY KEY (user_id)
);

DELIMITER $$
CREATE PROCEDURE sp_createUser(
  IN p_name VARCHAR(45), IN p_username VARCHAR(45), IN p_password VARCHAR(255))
BEGIN
  IF (SELECT EXISTS (SELECT 1 FROM tbl_user WHERE user_username = p_username)) THEN
    SELECT 'Username Exists !!';
  ELSE
    INSERT INTO tbl_user(user_name, user_username, user_password)
    VALUES (p_name, p_username, p_password);
  END IF;
END$$

CREATE PROCEDURE sp_validateLogin(IN p_username VARCHAR(45))
BEGIN
  SELECT * FROM tbl_user WHERE user_username = p_username;
END$$
DELIMITER ;
EOF

kubectl create configmap mysql-init -n flask-app --from-file=init.sql
```

Files in `/docker-entrypoint-initdb.d` run automatically the first time MySQL starts with an empty data directory.

5.3 MySQL manifest

```
cat > mysql.yaml <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  namespace: flask-app
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: managed-csi
  resources:
    requests:
      storage: 5Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  namespace: flask-app
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        ports:
        - containerPort: 3306
        envFrom:
```

```
        - secretRef:
            name: mysql-secret
        resources:
          requests: { cpu: 250m, memory: 512Mi }
          limits:   { cpu: 500m, memory: 1Gi }
        volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
          subPath: mysql
        - name: init
          mountPath: /docker-entrypoint-initdb.d
        readinessProbe:
          exec:
            command: ["sh", "-c", "mysqladmin ping -uroot -p$MYSQL_ROOT_PASSWORD"]
          initialDelaySeconds: 20
          periodSeconds: 10
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: mysql-pvc
      - name: init
        configMap:
          name: mysql-init
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: flask-app
spec:
  type: ClusterIP
  selector:
    app: mysql
  ports:
  - port: 3306
    targetPort: 3306
EOF

kubectl apply -f mysql.yaml
kubectl get pods -n flask-app -w
```

The `subPath: mysql` avoids the "lost+found" error on Azure Disks. `Recreate` strategy is needed because an Azure Disk can attach to only one pod at a time. Wait until the pod is `1/1 Running`, then press Ctrl+C.

Verify the DB:

```
kubectl exec -n flask-app deploy/mysql -- mysql -uroot -p'ChangeMe#123' \
  -e "USE BucketList; SHOW TABLES;"
```

Step 6: Deploy the Flask App

```
export ACR_LOGIN=$(az acr show -n $ACR_NAME --query loginServer -o tsv)

cat > app.yaml <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
  namespace: flask-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
      - name: flask-app
        image: ${ACR_LOGIN}/${IMAGE}:${TAG}
        ports:
        - containerPort: 5000
        env:
        - name: DB_HOST
          value: mysql
        - name: DB_USER
          value: root
        - name: DB_NAME
```

```
          value: BucketList
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysql-secret
              key: MYSQL_ROOT_PASSWORD
        resources:
          requests: { cpu: 100m, memory: 128Mi }
          limits:   { cpu: 500m, memory: 256Mi }
        readinessProbe:
          tcpSocket:
            port: 5000
          initialDelaySeconds: 5
          periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: flask-app-svc
  namespace: flask-app
spec:
  type: LoadBalancer
  selector:
    app: flask-app
  ports:
  - port: 80
    targetPort: 5000
EOF

kubectl apply -f app.yaml
kubectl get pods -n flask-app
```

ADJUST: the env var names must match what your `app.py` reads. The CPU `requests` are required, because the HPA calculates utilization as a percentage of them.

Step 7: Configure Pod Autoscaling (HPA)

```
cat > hpa.yaml <<'EOF'
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: flask-app-hpa
  namespace: flask-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: flask-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
EOF

kubectl apply -f hpa.yaml
kubectl get hpa -n flask-app
```

AKS ships with metrics-server, so no extra install is needed. The TARGETS column may show `<unknown>` for about a minute, then a real percentage.

This gives you two scaling layers:

Layer	Mechanism	Scales
Pods	HPA (CPU > 50%)	2 to 10 replicas
Nodes	Cluster autoscaler	2 to 4 nodes when pods are Pending

Step 8: Verify and Get the Final URL

```
kubectl get svc flask-app-svc -n flask-app -w
```

Wait until `EXTERNAL-IP` changes from `<pending>` to an IP (1-3 minutes), then press Ctrl+C.

```
EXTERNAL_IP=$(kubectl get svc flask-app-svc -n flask-app \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "Application URL: http://$EXTERNAL_IP"
curl -I http://$EXTERNAL_IP
```

Your final URL is `http://<EXTERNAL-IP>`, for example `http://20.xx.xx.xx`. Open it in a browser and test sign-up and login. Then confirm the data landed in MySQL:

```
kubectl exec -n flask-app deploy/mysql -- mysql -uroot -p'ChangeMe#123' \
  -e "SELECT user_id,user_username FROM BucketList.tbl_user;"
```

Step 9: Test Autoscaling

Terminal 1 (watch):

```
kubectl get hpa -n flask-app -w
```

Terminal 2 (generate load):

```
kubectl run load-gen -n flask-app --rm -it --image=busybox:1.36 --restart=Never -- \
  /bin/sh -c "while true; do wget -q -O- http://flask-app-svc >/dev/null; done"
```

Run several `load-gen` pods under different names if one isn't enough to push CPU past 50%. Within 1-3 minutes you should see CPU climb and REPLICAS grow from 2 toward 10:

```
kubectl get pods -n flask-app
kubectl top pods -n flask-app
```

If pods go `Pending` for lack of CPU, the cluster autoscaler adds nodes:

```
kubectl get nodes
```

Stop the load with Ctrl+C. After about 5 minutes, the HPA scales back down to 2.

Step 10: Updating the App (Optional)

```
az acr build -r $ACR_NAME -t $IMAGE:v2 .
kubectl set image deployment/flask-app flask-app=$ACR_LOGIN/$IMAGE:v2 -n flask-app
kubectl rollout status deployment/flask-app -n flask-app
```

Troubleshooting

Symptom	Fix
`ImagePullBackOff`	Run `az aks update -g $RG -n $AKS_NAME --attach-acr $ACR_NAME`, and check the image name and tag.
Flask pod `CrashLoopBackOff`	Run `kubectl logs -n flask-app deploy/flask-app`. Usually a wrong gunicorn target, a missing dependency, or the DB not ready yet.
App error "Can't connect to MySQL"	Check the DB env var names and that `DB_HOST=mysql`. Check `kubectl get pods`.
MySQL pod `Pending`	Run `kubectl describe pvc mysql-pvc -n flask-app` to check the storage class.
HPA shows `<unknown>`	Wait 1-2 minutes, and make sure CPU `requests` are set.
EXTERNAL-IP stuck on `<pending>`	Run `kubectl describe svc flask-app-svc -n flask-app` and check your regional public IP quota.

Symptom	Fix
Init SQL didn't run	It runs only on an empty volume. Delete the PVC and pod and redeploy to re-run it.

Useful commands: `kubectl describe pod <pod> -n flask-app` and `kubectl get events -n flask-app --sort-by=.lastTimestamp`.

Step 11: Cleanup

Delete everything to avoid charges:

```
az group delete -n $RG --yes --no-wait
```

Production Notes

This lab is not production-hardened. A real deployment would typically:

- Use **Azure Database for MySQL Flexible Server** instead of an in-cluster MySQL pod. A single pod is a single point of failure.
- Use **Azure Key Vault** with the CSI driver instead of Kubernetes secrets.
- Use an **Ingress controller** with TLS (cert-manager) instead of a bare LoadBalancer.
- Use a dedicated non-root DB user instead of `root`.